

Soutenance de thèse

Conception d'un système mémoire pour traitement de données creuses

Thèse soutenue par

Valentin ISAAC--CHASSANDE

le 11 mai 2026

Université Grenoble Alpes, CEA List, TiMA

PLAN

1. Les matrices creuses

2. Accélérateurs dédiées et Directions

3. Cache de réduction

4. SpDCCache : Fonctionnement et Architecture Matérielle

5. Validation de SpDCCache via un modèle analytique

6. Evaluations et Performances de SpDCCache

7. Conclusion

PLAN

1. Les matrices creuses

2. Accélérateurs dédiées et Directions

3. Cache de réduction

4. SpDCCache : Fonctionnement et Architecture Matérielle

5. Validation de SpDCCache via un modèle analytique

6. Evaluations et Performances de SpDCCache

7. Conclusion

LES MATRICES CREUSES

Applications : Physiques numériques, Intelligence artificielle, Analyse de graphes...

→ Matrice creuse

- **Larges** (nombre d'éléments non-nuls jusqu'à $\sim 10^9$)
- **Creuses** (densité jusqu'à $\sim 10^{-7}$)

→ **Compressées en mémoire** : Formats creux

- **Diverses**

- ▶ En tailles et densités
- ▶ En structures



[1] Zhiyao Li, Jiaxiang Li, Taijie Chen, Dimin Niu, Hongzhong Zheng, Yuan Xie, and Mingyu Gao. 2023. Spada : Accelerating Sparse Matrix Multiplication with Adaptive Dataflow. ASPLOS 2023. Association for Computing Machinery, 747–761.

LES FORMATS CREUX

Formats creux [2] [3] → pour éviter de stocker les zéros

- Beaucoup de formats spécifiques à des structures de matrices
- CSR et CSC → les plus simples et plus utilisées

CSR (Compressed Sparse Row)

CSC (Compressed Sparse Column)

[2] Reginald P. Tewarson. 1973. Sparse matrices. Vol. 69. Academic press New York, State University of New York, Stony Brook, NY.
[3] Yousef Saad. 2003. Iterative Methods for Sparse Linear Systems (second ed.). Society for Industrial and Applied Mathematics.

ÉCHANGER UN PROBLÈME POUR UN AUTRE

Formats creux :

→ **Empreinte mémoire réduite** de $\mathcal{O}(n^2)$ à $\mathcal{O}(n)$ → une nécessité

→ **Utilisation de métadonnées** → génèrent des transactions mémoires vers des adresses non-consécutives

Indirections

- **Accès indirects** depuis/vers la hiérarchie mémoire
- Du point de vue d'un système mémoire naïf : **accès « aléatoires »**

LES NOYAUX DE CACLUL (KERNELS)

Où apparaissent les indirections ?

Les matrices creuses sont utilisées dans de **nombreux kernels** :

- $1\mathcal{D}$ Multiplication **matrice** creuse-**vecteur** : SpMV
 - $2\mathcal{D}$ Multiplication **matrice-matrice** : SpMSPM, SpGEMM, SpMM
 - $n\mathcal{D}$ Les **tenseurs** en général
- } **Kernels de base** utilisés dans la majorité des applications

Chaque kernel peut être implémenté suivant **plusieurs algorithmes** :

- **Inner-Product** (IP) ($\geq 1\mathcal{D}$)
 - **Outer-Product** (OP) ($\geq 1\mathcal{D}$)
 - **Gustavson** (ou Row-wise) ($\geq 2\mathcal{D}$)
- } **Algorithmes de base**
- Des algorithmes plus complexes peuvent être vues comme des combinaisons de ces trois là.
- Méthodes de tiling → ~ algorithmes customisés / pré-traitement ($\geq 1\mathcal{D}$)

KERNELS CREUX – INNER-PRODUCT SpMV

Exemple de **SpMV** (multiplication matrice creuse-vecteur) : $A \times b = c$, avec A de taille (M, N) .

- $\forall (i, j) \in \llbracket 0, M-1 \rrbracket \times \llbracket 0, N-1 \rrbracket$, $c_i = c_i + A_{i,j} \times b_j$
- A est stockée avec un format creux comme CSR ou CSC
- Des **accès indirectes** apparaissent, sur les entrées ou la sortie, en fonction de l'algorithme :
 - ▶ **Inner-Product (IP)** :

$c_0 += A_{0,0} \times b_0$ (0)
 $c_0 += A_{0,0} \times b_0$ (0)
 $c_0 += A_{0,0} \times b_0$ (0)
 $c_0 += A_{0,0} \times b_0$ (0)
 $c_0 += A_{0,0} \times b_0$ (0)
 $c_0 += A_{0,0} \times b_0$ (0)
 $c_0 += A_{0,0} \times b_0$ (0)
 $c_0 += A_{0,0} \times b_0$ (0)
 $c_0 += A_{0,0} \times b_0$ (0)



list



list

$c_0 += VAL_0 \times b_{IDX_0}$ ①
 $c_0 += VAL_0 \times b_{IDX_0}$ ①
 $c_0 += VAL_0 \times b_{IDX_0}$ ①
 $c_0 += VAL_0 \times b_{IDX_0}$ ①

→ Avec le **SpMV** en **Inner-Product**, les accès « aléatoires » se produisent sur le vecteur d'entrée b , au moment de calculer la somme partielle $A_{i,j} \times b_j$.

KERNELS CREUX – OUTER-PRODUCT SPMV

...

KERNELS CREUX – GUSTAVSON SpMSpM

Exemple de **SpMSpM** (multiplication matrice creuse-matrice creuse) : $A \times B = C$, avec A de taille (M, K) , B de taille (K, N) , et C de taille (M, N) .

- $\forall (i, j, k) \in [0, M-1] \times [0, N-1] \times [0, K-1]$, $C_{i,j} = C_{i,j} + A_{i,k} \times B_{k,j}$
- A et B sont stockées avec un format creux comme CSR ou CSC
- Des **accès indirectes** apparaissent, sur les entrées ou la sortie, en fonction de l'algorithme :
 - ▶ **Gustavson (Gust)** :



list



list



➔ Avec le **SpMSpM** en **Gustavson**, les accès « **aléatoires** » se produisent sur les **lignes de la matrice d'entrée B** et dans les **lignes de la matrice de sortie C** , au moment d'accumuler la **somme partielle** $A_{i,k} \times B_{k,j}$ avec $C_{i,j}$.

➔ Gustavson se place comme un algorithme **intermédiaire** entre inner-product et outer-product

LA HIÉRARCHIE MÉMOIRE

Pourquoi les accès « aléatoires » sont un problème ?

Avec une matrice dense → localités spatiale et temporelle
Avec une **matrice creuse** → **pas de localité**

Dans une architecture classique

- Les hiérarchies mémoires prennent avantage des accès temporellement et spatialement locaux

Caches → **non-adaptés** aux accès « aléatoires »

- Cache remplie de zéros (lignes de cache remplies à 40% d'éléments non-nuls)
- Nombre de hits bas
- Évincements non-nécessaires → Beaucoup de **transactions mémoires** avec des **temps de réponse élevés**

Simple **préfetcher linéaire** → pas suffisant pour s'occuper du goulot d'étranglement mémoire

RÉSUMÉ

...

PROBLÉMATIQUE

Plus général (on sait pas encore que ce sera scatter focus)

PLAN

1. Les matrices creuses

2. Accélérateurs dédiés et Directions

3. Cache de réduction

4. SpDCCache : Fonctionnement et Architecture Matérielle

5. Validation de SpDCCache via un modèle analytique

6. Evaluations et Performances de SpDCCache

7. Conclusion

ACCÉLÉRATEURS MATÉRIELS DÉDIÉS

...

ACCÉLÉRATEURS MATÉRIELS DÉDIÉS

...

Matrices creuses
○○○○○○○○○○

État-de-l'art
○○●○○○○

Réductions
○○○○○

SpDCache
○○

Modèle analytique
○○○○○

Evaluations
○○○○

Conclusion
○○

GAMMA ? ? ?

...

Matrices creuses
○○○○○○○○○○○

État-de-l'art
○○○○●○○○

Réductions
○○○○○

SpDCache
○○

Modèle analytique
○○○○○

Evaluations
○○○○

Conclusion
○○

PHI ? ? ?

...

Matrices creuses
○○○○○○○○○○○

État-de-l'art
○○○○○●○○○

Réductions
○○○○○

SpDCache
○○

Modèle analytique
○○○○○

Evaluations
○○○○

Conclusion
○○

QUELLE EST LA MEILLEURE SOLUTION ?

...

Parmi les accélérateurs complets -> gust
Etendre la discussion sur avantatge OP sur IP
> 1 Gust 2 OP 3 IP

AVANT DE CONCEVOIR UN ACCÉLÉRATEUR POUR LE CALCUL CREUX

...

DIRECTION

Hypothèses :

OP SpMV

architecture générique, faible coût

structure des matrices

Approche : (résumé en qq mots)

PLAN

1. Les matrices creuses

2. Accélérateurs dédiées et Directions

3. Cache de réduction

- Cache générique
- Cache de réduction

4. SpDCache : Fonctionnement et Architecture Matérielle

5. Validation de SpDCache via un modèle analytique

6. Evaluations et Performances de SpDCache

7. Conclusion

CONTEXTE

Noyau de calcul : SpMV (multiplication
matrice creuse-vecteur)

Algorithme : OP (*Outer-Product*)

Avec OP SpMV, les accès « aléatoires » se
produisent sur le vecteur de sortie et sont des
réductions

➔ Réductions « aléatoires »

Une réduction sur un vecteur v :

$v_{indice} += valeur$

- Une lecture en mémoire de v_{indice}
- Une accumulation pour mettre à jour v_{indice} avec
 $valeur$
- Une écriture en mémoire de v_{indice} mis à jour

Cette opération est noté RED :

$RED(indice, valeur)$

SpMV : $A \times b = c$

Avec A une matrice creuse de taille (M, N) ,
 b et c des vecteurs denses de taille N et M

OP (*Outer-Product*)

```
for  $n \in [0, N - 1]$ 
  for  $m \in [0, M - 1]$ 
     $c_m += A_{m,n} \times b_n$ 
  end
end
```

➔ Accès séquentiels sur les entrées A et b

➔ Accès « aléatoire » sur le vecteur de sortie c

CACHE GÉNÉRIQUE

Granularité : ligne de cache (64 octets)

Transactions mémoires : lecture et écriture de ligne de 64 octets

Réduction : $c_{indice} += valeur$

Cas de hit :

- Lecture du cache ✓
- Accumulation dans le processeur
- Ecriture dans le cache ✓

Cas de miss :

- Lecture en mémoire et mise à jour du cache ✗
- Accumulation dans le processeur
- Ecriture dans le cache ✓

Cas de miss avec évincement :

- Ecriture en mémoire de la donnée évincée ✗
- Lecture en mémoire et mise à jour du cache ✗
- Accumulation dans le processeur
- Ecriture dans le cache ✓



<schéma d'une réduction dans un cache générique>

CACHE DE RÉDUCTION

Cache de réduction : cache générique avec quelques modifications

- Supporte des instructions de réduction RED
- Possède un accumulateur : accumulation dans le cache



<Figure comme SpDCCache (voir ASAP) ou on montre ce qu'il y a en plus (?)>

CACHE DE RÉDUCTION



list

<schéma d'une réduction dans un cache générique>



list

<schéma d'une réduction dans un cache de réduction>

Comparaison du nombre d'instruction / le processeur n'attend pas les REDs

GC 0 1 2

RedC 0 0 2

Evincement : le processeur n'attend pas de réponse

PLAN

1. Les matrices creuses

2. Accélérateurs dédiées et Directions

3. Cache de réduction

4. SpDCCache : Fonctionnement et Architecture Matérielle

5. Validation de SpDCCache via un modèle analytique

6. Evaluations et Performances de SpDCCache

7. Conclusion

SPDCACHE

4 slides spdcache de ASAP

PLAN

1. Les matrices creuses

2. Accélérateurs dédiées et Directions

3. Cache de réduction

4. SpDCCache : Fonctionnement et Architecture Matérielle

5. Validation de SpDCCache via un modèle analytique

6. Evaluations et Performances de SpDCCache

7. Conclusion

MULTIPLICATION MATRICE-VECTEUR DANS UN CACHE GÉNÉRIQUE :

Matrice quelconque :

- Évincements répétés dans le cache
- Le cache n'est pas utile ✖

Petite matrice, bande horizontale :

- Tiens dans le cache, pas d'évincement
- Le cache est utile ✔
- Cas peu intéressants ✖

Matrice bandée :

- Évincements répétés dans le cache
- Le cache n'est pas utile ✖
- Structure très courante ✔

Matrice parfaitement bandée :

- Nombre d'évincements limité
- Le cache est utile ✔
- Structure peu courante ✖



list

TRAFFIC MÉMOIRE THÉORIQUE AVEC DES MATRICES BANDÉE

C

Traffic mémoire du vecteur de sortie
versus

densité en dehors de la bande,
pour différentes tailles de cache

- Densité dans la bande fixe (10^{-2})
-
- ➔ Réduction de deux ordres de grandeur
 - SI Matrice parfaitement bandée
 - ET Largeur de bande suffisamment petite par rapport à la taille du cache

<relation>



(graph avec courbes qui apparaissent au bon moment)

CONCLUSION

On sélectionne une bande diagonale de la matrice, qu'on traite avec un cache set-associatif

- <relation>

QUE FAIRE DU RESTE, HORS DE LA BANDE ?

Granularité : Éléments isolés

- Incompatible avec les lignes de cache

➔ Pas de ligne de cache :

64o (8 mots) → 8o (1 mot)

Absence de structure claire

- Nombre de *hits* bas

➔ Pour augmenter la quantité de *hits*
dans un espace de cache modeste :

Cache complètement associatif

Transactions mémoires inadaptées

- Lectures et écritures sur 64o (8 mots)

➔ Transactions mémoires "par éléments"
RMW atomique sur 8o (1 mot)



(graph avec courbes qui apparaissent au bon moment)

CONCLUSION

On traite les éléments hors-bande de la matrice avec un cache « par élément », complètement associatif, avec des transactions mémoires « par élément » (RMW)

CONCLUSION

...

PLAN

1. Les matrices creuses

2. Accélérateurs dédiées et Directions

3. Cache de réduction

4. SpDCCache : Fonctionnement et Architecture Matérielle

5. Validation de SpDCCache via un modèle analytique

6. Evaluations et Performances de SpDCCache

7. Conclusion

ENVIRONNEMENT

CVA6 + SpDCache + RAM

TRAFFIC MÉMOIRE

Set de 48 à 62 matrices

Toutes structures

Config 25/50/25

Perfs avec écart type (brut et ×)

Conclu : réduction significative du trafic mémoire

ACCÉLÉRATION

1 coeur CVA6, basse perf -> on est dans une zone entre compute et memory bound

Permet de clasifier les marices selon nouveau critere dcd

Explique les deux groupes d'accélération

Conclu : Comme attendu, reduction du trafic permet accélération sur matrice memory bound /
classification des matrices selon densité des lignes de cache

PLAN

1. Les matrices creuses

2. Accélérateurs dédiées et Directions

3. Cache de réduction

4. SpDCCache : Fonctionnement et Architecture Matérielle

5. Validation de SpDCCache via un modèle analytique

6. Evaluations et Performances de SpDCCache

7. Conclusion

CONCLUSION

...

Thanks!

contact

backup slides :

Exploration des paramètres (Python)

BaCSC

Toutes les perfs

Perf Python